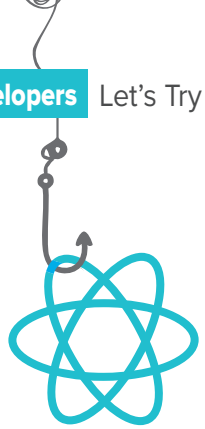

Hooked on React Hooks!

In this final part of the two-part in-depth series on React Hooks, we continue to discuss a few more hooks and custom hooks.

In the first article in the series, which was carried in the October 2022 issue of *OSFY*, we learnt what hooks and custom hooks are, and discussed a few hooks briefly. We will look at a few more hooks and custom hooks in this second and final article on React Hooks.

useImperativeHandle

```
useImperativeHandle(ref, createHandle, [deps]);
```

```
function Button (props, ref) {
    const buttonRef = useRef();
    useImperativeHandle(ref, () => ({
        focus: () => {
            console.log ("Inside focus");
            buttonRef.current.focus();
        }
    }));
    return (
        <button ref={buttonRef} {...props}>
            Button
        </button>
    );
}
export default forwardRef (Button);
```

Usually, when you use *useRef*, you are given the instance value of the component the ref is attached to. This allows you to interact with the DOM element directly. *useImperativeHandle* is very similar but it lets you do two things -- it gives you control over the value that is returned; so instead of returning the instance element, you explicitly state what the return value will be. Also, it allows you to replace native functions such as *blur* and *focus* with functions of your own, thus allowing side effects to the normal behaviour or different behaviour altogether. And the best point is you can call the function whatever you like. So, basically, *useImperativeHandle* customises the instance value that is exposed to parent components when using ref.

In the example above, the value we get from ref will only contain the function *focus*, which we declared in *useImperativeHandle*. Also, you can customise the function to behave in a different way than you would normally expect. So, for example, in the *focus* function, we can update the document title or log as seen in the code but notice that we have used *forwardRef*. In React, *forwardRef* is a method that allows parent components to pass down *ref* to their children, and it gives a child component a reference to a DOM element, which is created by its parent component.

So, basically, *useImperativeHandle* is used when you might not want to expose native properties to the parent or (as in our case) want to change the behaviour of a native function. As in the case of additional hooks, you'll rarely use *useImperativeHandle* in your code base, but it's good to know how to use it.

useLayoutEffect

```
const ref = React.useRef()
React.useEffect (() => {
    ref.current = 'Lorem ipsum'
})

React.useLayoutEffect (() => {
    console.log(ref.current) // This runs first, will log old
value
})
```

As we saw with the *useEffect*, the code inside of *useLayoutEffect* is fired after the layout is rendered. Although this makes it suitable for, let's say, setting up subscriptions and event handlers, not all effects can be deferred. So, basically, we should understand the difference between passive and active event listeners. Let's say we want to make changes to the DOM synchronously before we render the additional components. If we use *useEffect*, this can lead to inconsistencies in the UI. As a result, for these types of events, React provides us with *useLayoutEffect*. It has the same signature as *useEffect* but the only difference is when it is fired. Your code runs immediately after the DOM has been updated but before the browser has had a chance to paint those changes.